

The Prompt Is Not the Program: Execution Lineage and the New Unit of Prompt Practice

Working Paper · 2026-08-17
SLIPCASE-20260817T2039-0400-304d480a176f

Abstract

Prompt research often treats the visible prompt as the causal unit linking an author to a generated artifact. The field compiled here makes that unit unstable. Text-to-image systems can return polished outputs while failing to interpret parts of the input; deployed systems can rewrite a user’s request before generation; prompt compilers can synthesize the executed wording from declarations and metrics; soft prompts can control models without readable words; and optimization systems can revise or evolve prompts without preserving any privileged original sentence. I argue that the durable unit of prompt practice is therefore not the prompt but the execution lineage: the traceable sequence from user formulation through transformations, conditioning, sampling, output, evaluation, and revision. This shift changes what must be preserved for reproducibility, authorship, debugging, and criticism. A prompt receipt that stores only visible text is insufficient whenever the executed instruction, model state, or selection machinery differs from what the user wrote. The paper proposes a minimal execution-lineage schema and shows how default-image failures make the stakes visible: successful generation and successful interpretation are separable events.

1 The wrong unit

The prompt is attractive as a unit because it is visible, copyable, and apparently causal. A user writes a sentence, a model returns an artifact, and the interface places the two next to each other. That adjacency encourages a source-code picture: the prompt looks like the instruction that executed. The research field assembled here repeatedly breaks that picture. Liang et al. describe prompt programming as a practice in which requirements and implementation can partially occupy the same textual artifact (Liang et al., 2025). LMQL surrounds natural-language instructions with formal control flow and constraints (Beurer-Kellner et al., 2023). DSPy goes further by treating prompt wording as something a compiler can optimize from declarations and metrics (Khattab et al., 2023). In each case, ‘the prompt’ names a different layer. The central question is therefore not whether prompts are programs, but which prompt-like object is actually doing causal work in a given system.

2 Failure without an error state

Simonen et al.’s study of Midjourney default images supplies the decisive counterexample to equating successful execution with successful interpretation (Simonen et al., 2026). The system always returns an image, including for terms it may not recognize. Dissimilar prompts can converge on recurrent motifs. The interface therefore reports success at the computational level while the output may reveal semantic failure. Their ablations add two complications. Recognized terms inside a longer prompt can suppress visible default behavior, hiding the failure of an unknown term; and seed changes can alter which recurrent motif appears. A visible result is thus jointly shaped by unequal semantic control and sampling state. The paper itself is careful about causality: because Midjourney is proprietary, the recurrence might reflect training-data gaps, text representation, prompt rewriting,

sampling, or interactions among these layers. That causal opacity is not a side issue. It is evidence that the visible prompt-output pair is an incomplete experimental record.

3 The hidden writer

The possibility of prompt rewriting is architecturally ordinary, not exotic. OpenAI’s DALL-E 3 documentation describes a pipeline in which ChatGPT expands a user’s idea into a more detailed prompt before image generation (OpenAI, 2023). In such a system, the sentence the user authored and the sentence that conditioned the image generator are different objects. Two distinct user prompts can in principle converge before the image model ever sees them. Default-like output convergence could therefore be produced by rewrite collapse upstream of generation, by generative collapse downstream, or by both. DSPy makes a related separation explicit in programming practice: a developer specifies modules and objectives while a compiler searches for prompts and demonstrations that work (Khattab et al., 2023). The prompt becomes an intermediate representation or build artifact. Once that happens, copying the user-visible prose is no longer equivalent to preserving the executable instruction.

4 Control after words

Soft prompting removes an even deeper assumption. Qin and Eisner show that trainable continuous vectors can condition a language model effectively without corresponding to readable words (Qin and Eisner, 2021). Milliere’s made-up-word experiments show the complementary case: a string meaningless to human lexical practice can still be model-significant because tokenization and multilingual fragments activate learned associations (Milliere, 2022). Struppek et al. show that a single homoglyph can shift visual outputs through text-encoder geometry (Struppek et al., 2023). These results separate the human-readable string from the model-relative control signal. Natural language remains special not because it is the only way to condition a model, but because it can be simultaneously legible to a person and operative for a machine. Prompt practice occupies that unstable overlap.

5 Revision becomes machinery

Automatic prompt optimization turns revision itself into an executable process. Pryzant et al. generate natural-language criticisms of prompt failures, call them textual gradients, and use those critiques to propose new prompt candidates (Pryzant et al., 2023). Promptbreeder maintains populations of task prompts and evolves the mutation prompts that generate further mutations (Fernando et al., 2023). Here the author no longer selects every sentence. Control migrates toward objectives, examples, mutation rules, and fitness functions. The resulting lineage can contain many prompts with no single wording privileged as the program. If a prompt is optimized anew for a different model or metric, the durable artifact may be the search procedure plus evaluation regime rather than any one textual result.

6 What the prompt does not say

Underspecification further enlarges the operative system beyond the words present. Yang et al. report that models sometimes satisfy omitted requirements, but such behavior is less stable, while specifying

more requirements can itself degrade instruction following (Yang et al., 2025). The executed behavior is therefore supplied jointly by written instructions, model defaults, context, examples, interface conventions, and post-training. More written specification does not monotonically increase control. For creative practice this is especially consequential: omission can be defect, delegation, or productive aperture. A provenance record must distinguish what was explicitly requested from what the system supplied by default, because both can shape the artifact.

7 Execution lineage

The field therefore suggests a different unit. Call it execution lineage. A minimal lineage records: (1) user formulation; (2) any rewrite, compilation, retrieval, or prompt assembly; (3) the effective conditioning object, when inspectable; (4) model and version; (5) stochastic and generation parameters; (6) generated candidates; (7) evaluation or selection criteria; and (8) revisions that lead to the next run. ChainForge already points toward this contrastive view by treating prompts as variables in a field of model and prompt comparisons rather than isolated messages (Arawjo et al., 2023). The default-image study supplies a parallel methodological lesson: a failure becomes visible only by comparing relations among many prompts with relations among many outputs (Simonen et al., 2026). A chronological transcript is therefore not enough. Reproducibility requires counterfactual and transformational structure.

8 A minimal receipt

A practical prompt receipt should preserve at least `SOURCE_TEXT`, `EXECUTED_TEXT_OR_STATE`, `TRANSFORMATIONS`, `MODEL_ID`, `MODEL_VERSION`, `PARAMETERS`, `RANDOM_STATE_WHEN_AVAILABLE`, `OUTPUT_SET`, `SELECTION_RULE`, `EVALUATOR`, and `LINEAGE_PARENT`. Some fields will often be unavailable in proprietary systems. That absence should be represented explicitly rather than filled by inference. The point is not to demand impossible transparency from every interface. It is to stop calling a partial record complete. The distinction mirrors Simonen et al.’s causal caution: observable recurrence can be real even when its location inside a black-box pipeline remains unresolved (Simonen et al., 2026).

9 Consequences

This shift alters four familiar questions. For authorship, the question moves from ‘who wrote the prompt?’ to ‘who or what selected the transformations and evaluative conditions that produced the executed lineage?’ For debugging, a bad output cannot be localized to prompt wording until upstream rewriting, downstream conditioning, and sampling have been ruled out. For criticism, the visible prompt may be an unreliable textual object if a compiler or optimizer produced the effective instruction. For reproducibility, preserving only the final successful prompt can erase the path-dependent evidence that made its apparent success intelligible. The archive must preserve failures and alternatives as evidence, not merely the winning sentence.

10 Limits and tests

The argument is deliberately stronger about provenance than about mechanism. Most of the technical mechanisms in this field come from different model families and should not be merged into

one causal story. A-STAR shows that concept-related attention can be lost across diffusion timesteps (Agarwal et al., 2023), Mao et al. show that initial latent noise can exhibit content preferences in Stable Diffusion (Mao et al., 2023), and Chowdhury et al. show that CLIP-like retrieval spaces can develop hubs (Chowdhury et al., 2024). None of these results proves the mechanism behind Midjourney’s default images. Their value here is oppositional: each supplies another plausible location at which the naive visible-prompt-to-output story can break. The next empirical program should therefore instrument an open pipeline and record the lineage end to end while varying semantic familiarity, prompt rewriting, seed, encoder representation, and evaluation. The wager of this paper is testable: if the same visible prompt can yield systematically different executed representations or if distinct visible prompts converge before generation, then the prompt alone cannot serve as the reproducible program record.

11 Conclusion

Prompt practice began in interfaces that made writing and execution appear adjacent. The emerging technical ecology separates them. Prompts are rewritten, compiled, optimized, vectorized, mutated, selectively omitted, and probabilistically interpreted. Meanwhile, generative systems can produce fluent or beautiful artifacts even when parts of the instruction have ceased to control the result. The useful response is not to deny that prompts can be programmatic. It is to identify the larger program they participate in. For research, design, and archival practice, that program is best approached as an execution lineage. The prompt remains important, but it is one event in a chain whose transformations, defaults, stochastic states, and evaluative choices must be preserved if words are to function as inspectable operations rather than retrospective folklore.

Execution-lineage receipt: proposed minimum

Field	Purpose
Source text	What the human or upstream system supplied.
Executed text or state	What actually conditioned the model, when inspectable.
Transformations	Rewrite, compile, retrieval, assembly, or optimization steps.
Model / version	The executing model identity and relevant release.
Parameters / random state	Sampling and generation state needed to interpret variance.
Output set	Candidates, not only the selected artifact.
Selection / evaluator	Why one result or revision survived.
Lineage parent	The prior receipt from which this run descended.

Source and verification note

This working paper was compiled from 21 zettels preserved in the current research field. The Simonen et al. CHI 2026 paper is present locally in the package and was inspected directly. Other cited works are preserved as source records and link-only receipts from the zettels; their bodies were not independently retrieved during this build. The paper’s source map states the card and citekey behind each major claim. The assembly prompt and verification report are included in the package.

References

- Agarwal, A., Karanam, S., Joseph, K. J., Saxena, A., Goswami, K., and Srinivasan, B. V. (2023). A-STAR: Test-time Attention Segregation and Retention for Text-to-image Synthesis. ICCV. <https://arxiv.org/abs/2306.14544>
- Arawjo, I., Swoopes, C., Vaithilingam, P., Wattenberg, M., and Glassman, E. L. (2023). ChainForge: A Visual Toolkit for Prompt Engineering and LLM Hypothesis Testing. <https://arxiv.org/abs/2309.09128>
- Beurer-Kellner, L., Fischer, M., and Vechev, M. (2023). Prompting Is Programming: A Query Language for Large Language Models. PLDI. DOI: 10.1145/3591300.
- Chowdhury, N., Wang, F., Shenoy, S., Kiela, D., Schwettmann, S., and Thrush, T. (2024). Nearest Neighbor Normalization Improves Multimodal Retrieval. <https://arxiv.org/abs/2410.24114>
- Fernando, C., Banarse, D., Michalewski, H., Osindero, S., and Rocktaschel, T. (2023). Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution. <https://arxiv.org/abs/2309.16797>
- Khattab, O. et al. (2023). DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. <https://arxiv.org/abs/2310.03714>
- Liang, J. T., Lin, M., Rao, N., and Myers, B. A. (2025). Prompts Are Programs Too! Understanding How Developers Build Software Containing Prompts. Proceedings of the ACM on Software Engineering. DOI: 10.1145/3729342.
- Mao, J., Wang, X., and Aizawa, K. (2023). Guided Image Synthesis via Initial Image Editing in Diffusion Model. <https://arxiv.org/abs/2305.03382>
- Milliere, R. (2022). Adversarial Attacks on Image Generation With Made-Up Words. <https://arxiv.org/abs/2208.04135>
- OpenAI. (2023). DALL-E 3. <https://openai.com/index/dall-e-3/>
- Pryzant, R., Iter, D., Li, J., Lee, Y. T., Zhu, C., and Zeng, M. (2023). Automatic Prompt Optimization with “Gradient Descent” and Beam Search. <https://arxiv.org/abs/2305.03495>
- Qin, G. and Eisner, J. (2021). Learning How to Ask: Querying LMs with Mixtures of Soft Prompts. <https://arxiv.org/abs/2104.06599>
- Simonen, H., Kiviniemi, A., Johnston, H., Barranha, H., and Oppenlaender, J. (2026). An Exploration of Default Images in Text-to-Image Generation. CHI ’26. DOI: 10.1145/3772318.3790681.
- Struppek, L., Hintersdorf, D., Friedrich, F., Brack, M., Schramowski, P., and Kersting, K. (2023). Exploiting Cultural Biases via Homoglyphs in Text-to-Image Synthesis. Journal of Artificial Intelligence Research 78, 1017–1068. <https://arxiv.org/abs/2209.08891>
- Yang, C., Shi, Y., Ma, Q., Liu, M. X., Kastner, C., and Wu, T. (2025). What Prompts Don’t Say: Understanding and Managing Underspecification in LLM Prompts. <https://arxiv.org/abs/2505.13360>